ATELIER MAILLEUR

Comprendre, utiliser et partager Mailleur

Fonctionnement, architecture, apprentissage, API et diffusion

Version 1.0.0rc5 — révision locale

4 novembre 2026
12 chapitres et 3 annexes
Mesures sur Apple M3, 16 Go

Guide destiné à comprendre le projet, démarrer sur cet ordinateur, utiliser l'API et préparer une diffusion reproductible.

Sommaire

1. Version étudiée et réponse pratique . . . 3

2. Ce que fait le logiciel . . . 4

3. Architecture et circulation des données . . . 5

4. Trois réseaux et plusieurs sens du mot modèle . . . 7

5. Corpus, entraînement et coût d'un million de cas . . . 8

6. Tailles, mémoire et partage . . . 9

7. Installer et commencer . . . 11

8. Fonctionnement de l'API . . . 12

9. Optimisations appliquées et améliorations suivantes . . . 13

10. Préparer GitHub et contribuer . . . 14

11. Petit glossaire . . . 15

12. Références et preuves . . . 16

Annexe A — Mesures sur Apple M3 . . . 17

Annexe B — Utiliser l'API . . . 22

Annexe C — Tailles et capacités . . . 31

Les liens du sommaire et les signets du PDF permettent de rejoindre les chapitres. Les fichiers Markdown restent les sources éditables de référence ; les annexes regroupent les mesures, les exemples d'API et l'inventaire des tailles.

1. Version étudiée et réponse pratique

Ce guide décrit **Atelier Mailleur 1.0.0rc5, révision locale du 4 novembre 2026**. Les sources ont été récupérées depuis le chat « Optimiser le mailleur », dans le projet « Créations de mailleur ». L'archive initiale a été vérifiée contre son manifeste SHA-256 : 1 326 fichiers vérifiés, ZIP de 20 319 571 octets. Les résultats Linux déjà livrés sont conservés ; les nouvelles mesures sur Apple M3 sont rangées dans `verification/local_m3/`.

Le dossier actif est `atelier_mailleur/`. Le dossier « mailleur automatique » présent auparavant sur cet ordinateur ne contenait aucun code. L'ancienne synthèse « Mailleur v2.5 » décrit une architecture différente et précise qu'elle a été reconstituée après effacement de son espace de travail. Le README de la rc5 indique que l'archive Rust d'origine n'a pas été retrouvée. Les annonces de GNN, PPO, tout-hexa et protocole JSON-lines de cette ancienne synthèse ne décrivent pas les sources récupérées ici.

Le MacBook Pro Apple M3 à 16 Go convient au démarrage, aux maillages de démonstration et à l'entraînement des petits réseaux de cette version. Son GPU à 10 cœurs n'est pas utilisé par l'entraîneur actuel. La production des maillages d'apprentissage, bien plus que la taille des poids, détermine le coût d'une grande campagne. Une compatibilité matérielle ne garantit pas qu'une pièce industrielle donnée tiendra dans 16 Go.

La machine disposait de Python 3.9.6, incompatible avec les exigences du projet, et d'aucune chaîne Cargo. Python 3.12 et Rust ont été installés dans des dossiers isolés du projet pour les vérifications. Le système et le profil du terminal ne sont pas modifiés. Le détail mesuré de l'environnement et les résultats figurent dans `verification/local_m3/` et dans le document `MESURES_M3.md`.

2. Ce que fait le logiciel

Mailleur prépare une géométrie, choisit des réglages de maillage, génère des éléments, contrôle leur qualité et écrit des fichiers destinés à un solveur. Il peut créer des primitives ou importer une pièce CAO. Le mode plan traite un domaine 2D, le mode surface une peau, et le mode volume un solide.

Les surfaces sont maillées en triangles, avec possibilité de recombinaison en quadrilatères. Les volumes sont maillés en tétraèdres. La version ne fournit pas un mailleur automatique tout-hexa pour des volumes arbitraires. Les éléments d'ordre 2 sont possibles selon le type et le format de sortie réellement pris en charge.

La « recherche » essaie plusieurs recettes de maillage : algorithme, tailles et réglages locaux. Elle retient la meilleure recette admissible parmi celles essayées dans le budget. Elle ne prouve pas que le maillage est optimal parmi toutes les solutions possibles.

Le logiciel peut mémoriser une recette réussie et la réexécuter sur une pièce identifiée. Le réseau prédit l'intérêt de recettes et peut contribuer à leur classement lorsqu'un gain de validation est démontré. Il ne dessine pas directement chaque nœud et ne remplace pas le moteur géométrique.

Les zones « contact », « interface » ou « contrainte » imposent des exigences locales de qualité. Elles doivent être désignées à partir du problème physique. Le programme ne calcule ni le champ de contraintes, ni les matériaux, ni les charges, ni une estimation d'erreur éléments finis. Un bon indicateur de forme ne dispense donc pas d'une étude de convergence dans le solveur.

3. Architecture et circulation des données

```
Utilisateur ou programme de calcul
            |
      Interface Qt / CLI / API HTTP (Python)
            |
      NativeSession : requête JSON sur disque
            |
      mailleur-core --request native_request.json (Rust)
            |
      API C de Gmsh -> OpenCASCADE / algorithmes natifs C++
            |
      géométrie, maillage, qualité, recherche, apprentissage
            |
      report.json + preview.json + mesh.msh + model.brep
            |
      Python : affichage, fichiers, conversion/export, réponse API
```

Couche	Fichiers principaux	Responsabilité
Interface de bureau	`atelier/desktop.py`, `viewport.py`, `desktop.qss`	Fenêtres, formulaires, caméra, sélection, rendu et affichage des rapports
Commandes	`atelier/__main__.py`	Installation diagnostiquée, génération, apprentissage, corpus et serveur
API HTTP locale	`atelier/server.py`, `worker.py`, `web/`	File de travaux, processus isolés, suivi, annulation et téléchargements
Pont vers Rust	`atelier/native.py`	Normalisation, copie de sources, requêtes, délai, lecture des résultats
Moteur	`rust_core/src/backend.rs`, `gmsh.rs`	Géométrie, unités, maillage, remaillage et appels natifs
Contrôles	`quality.rs`, `metrics.rs`	Qualité complète, zones, compteurs et métriques
Apprentissage	`advisor.rs`, `learning.rs`, `experience*.rs`, `dataset_model.rs`	Prédiction, entraînement, sélection et mémoire
Corpus	`corpus.rs`, `corpus_generator.rs`, `representation.rs`	Configurations reproductibles, segments, reprise et 128 caractéristiques
Exports	`atelier/formats.py`, `convert_worker.py`, `mailleur/io.py`	Écriture et conversion via meshio/h5py ou adaptateurs ; unités numériques dans Rust

Python effectue aussi les calculs nécessaires à la caméra et à l'affichage. Les convertisseurs réalisent les opérations de représentation et d'entrée/sortie. La séparation des responsabilités ne signifie donc pas « aucune opération arithmétique dans Python » : le calcul de géométrie, de maillage, de qualité et des réseaux appartient au noyau Rust et aux bibliothèques natives qu'il pilote.

Gmsh est chargé dynamiquement par `libloading`, via son API C. Le binaire Rust et la bibliothèque doivent correspondre au système et à l'architecture. Le petit binaire Linux x86_64 livré dans l'archive n'est pas exécutable sur Apple M3 ; le noyau doit être compilé pour `arm64`. `ATELIER_CORE` permet de choisir un noyau et `GMSH_LIBRARY` une bibliothèque explicite.

Parcours d'une génération

1. Le projet JSON indique géométrie, mode, unités, tailles, critères et zones. Une CAO importée reste un fichier source séparé ou embarqué dans un projet portable `.amp`.
2. Python vérifie la structure, copie la source dans la session et écrit `native_request.json`.

3. Rust valide les valeurs numériques, convertit les coordonnées en mètres internes et prépare la CAO avec Gmsh/OpenCASCADE. Une réparation éventuelle utilise une tolérance explicite.
4. Le backend applique les groupes, les champs de taille et les correspondances demandées, puis lance le maillage.
5. Rust contrôle l'ensemble des éléments, notamment les Jacobiennes et la qualité de forme, ainsi que les exigences des zones. L'aperçu peut être échantillonné ; les compteurs ne se limitent pas à cet aperçu.
6. Le moteur écrit rapport et artefacts. Python les affiche ou les renvoie. Un maillage refusé ne devient pas exportable simplement parce qu'un fichier intermédiaire existe.
7. L'export convertit les mètres internes vers l'unité demandée, écrit le format cible et conserve un rapport. L'export Cast3M s'appuie sur un flux UNV et un script d'import ; il ne définit pas à lui seul un calcul mécanique complet.

La marge autour d'une zone repose sur une boîte englobante élargie : c'est une sélection conservatrice, qui peut couvrir davantage qu'une distance exacte à la CAO. Le remaillage local travaille sur les entités prises en charge et vérifie la conservation hors sélection ; il ne fournit pas une reconstruction arbitraire d'un sous-volume.

4. Trois réseaux et plusieurs sens du mot modèle

Un **modèle CAO** est une pièce. Une **configuration** ajoute des tailles, des unités et des objectifs à cette pièce. Une **stratégie** est un essai de réglages pour une configuration. Un **modèle entraîné** est un petit fichier de poids et de métadonnées. Le corpus contient les exemples ; les poids en sont une synthèse numérique, sans inclure tous les maillages.

Réseau	Architecture	Paramètres	Usage
Conseiller historique	8 entrées, 12 neurones cachés, 3 sorties	147	Choisir entre trois stratégies du premier parcours de démonstration
Guide de mémoire interactive	128 entrées, 12 neurones cachés, 1 sortie	1 561	Estimer l'intérêt d'une recette dans la recherche sur une pièce
Guide du corpus	128 entrées, 32 neurones cachés par défaut, 1 sortie	4 161	Classer les recettes à partir du corpus segmenté

Le noyau utilise du calcul CPU en double précision `f64`. Pour le guide 128→32→1, le compte est `128×32 + 32 + 32 + 1 = 4161`. Les poids bruts occupent 33 288 octets ; le JSON contient en plus les nombres sous forme de texte, les métriques et la provenance. La largeur du réseau corpus est configurable dans l'entraîneur natif ; le tableau décrit le réglage livré.

Les 128 caractéristiques résument des dimensions, modes, topologies et informations spatiales ainsi que la recette proposée. Il s'agit d'une représentation compacte, pas d'une description CAO complète ni d'un graphe GNN. Deux formes différentes peuvent rester difficiles à distinguer avec ces descripteurs.

Les réseaux de démonstration sont sauvegardés mais leur classement reste désactivé lorsque la validation ne montre pas de progrès. Dans l'état final des démonstrations initiales livrées, les trois guides sont désactivés (`deploy_enabled=false`). Le pilote de corpus ne démontre pas de gain de sélection. La recherche classique et le réemploi exact des recettes continuent de fonctionner. Activer manuellement un modèle pour afficher une « IA active » ferait perdre cette protection méthodologique.

Recherche interactive et mémoire

`learn` examine une pièce, essaie un nombre borné de recettes, contrôle chacune, conserve la meilleure admissible et enregistre les réussites et les échecs. Le contexte inclut le contenu de la source et les paramètres, dont les critères de qualité. Le nom du fichier ne suffit pas à identifier une pièce.

`experience/memory.json` contient l'index ; `events.jsonl` conserve les événements ; `model.json` et ses versions conservent les poids. Une recette retrouvée est recalculée et contrôlée. L'index est limité à 1 000 contextes, 32 000 observations et 128 Mio ; le journal est limité à 512 Mio. Les limites provoquent un diagnostic, sans purge automatique. Cette mémoire n'est pas le stockage d'un million de configurations.

5. Corpus, entraînement et coût d'un million de cas

Le plan par défaut prévoit 250 000 configurations structurées et 750 000 aléatoires, réparties sur **24 familles**. Il n'exécute aucun de ces calculs lorsqu'on crée le plan. Les paramètres structurés suivent une répartition contrôlée ; les variantes aléatoires sont reproductibles à partir de la graine, de la famille et de l'indice, avec stratification marginale des paramètres.

Les familles couvrent plaques percées ou multi-trous, congés, chanfreins, équerres et polygones concaves, parois minces, sphères, boîtes, cadres, volumes perforés, bords minuscules et défauts invalides. La liste exacte et les règles sont dans `rust_core/src/corpus_generator.rs`. Il existe volontairement des cas invalides : enregistrer leur refus est utile, mais ils ne deviennent pas des maillages réussis.

Les familles sont réparties entre apprentissage (16), validation (4) et test (4). Les variantes d'une même famille restent dans la même partition. Les gradients utilisent uniquement l'apprentissage ; la validation sélectionne les poids et leur activation ; le test reste une évaluation distincte. Un million de variantes de ces 24 familles ne prouve pas la couverture de toutes les pièces industrielles.

Le pilote initial livré a réellement traité 48 configurations et 96 stratégies : 46 cas ont produit un maillage et 45 disposent d'au moins une stratégie admissible. Ces compteurs ne sont ni un million de maillages ni une preuve de généralisation. Les 33 010 cas synthétiques des benchmarks d'entraînement servent à tester les calculs et les fichiers JSONL ; leurs maillages n'ont pas été générés.

Les résultats vont dans `shards/cases-*.jsonl`. Chaque ligne contient une configuration et les observations de ses stratégies. Les maillages détaillés ne sont pas tous conservés par défaut. La reprise vérifie les empreintes et le curseur ; une altération de données durables est signalée. `--retain-meshes` augmente fortement le stockage.

L'entraînement lit progressivement les segments et forme des mini-lots. Il ne charge pas un million de maillages en RAM. Les poids et points de reprise sont distincts : les premiers servent à prédire ; les seconds incluent l'état nécessaire pour reprendre un apprentissage compatible. Les budgets de temps sont vérifiés entre étapes ; les contrôles d'intégrité et un appel Gmsh en cours ne deviennent pas instantanés à l'échéance.

Prévoir les durées sans inventer une vitesse

Le coût est approximativement `N × (inspection + somme des stratégies + fichiers)`, auquel s'ajoutent les reprises et l'entraînement. Avec quatre stratégies, un million de configurations représente jusqu'à quatre millions d'essais, plus les inspections. Le temps par essai dépend fortement de la géométrie et de la finesse.

Hypothèse illustrative par essai	Quatre millions d'essais séquentiels
0,1 seconde	4,63 jours
1 seconde	46,30 jours
5 secondes	231,48 jours

Ces valeurs sont des scénarios arithmétiques, hors inspections et écritures, pas des prévisions du Mac. Augmenter le nombre de processus ne divise pas automatiquement le temps par le nombre de cœurs : RAM, température, accès disque et bibliothèques natives interviennent. Sur 16 Go, commencer avec un seul travail simultané et un lot de 20 à 48 cas permet de mesurer le besoin réel.

6. Tailles, mémoire et partage

L'archive initiale occupe **20,32 Mo compressés** et **74,32 Mo décompressés**. Environ 70,30 Mo proviennent des preuves et historiques `verification/`, soit 94,6 % de la livraison. Le code spécifique est petit : environ 238 ko de Python applicatif, 76 ko de scripts et 508 ko de sources Rust, avant cette révision. Un environnement d'exécution inclut aussi Python, Qt, Gmsh, NumPy et les convertisseurs ; le dossier de développement inclut en plus Rust et son cache de compilation.

Objet livré dans les preuves initiales	Taille du fichier observé	À transmettre pour utiliser les poids ?
Guide historique JSON	13 869 octets	Seulement pour le parcours historique
Guide interactif récent JSON	49 964 octets	Pour transporter le réseau ; insuffisant seul pour reprendre l'apprentissage
Guide corpus récent JSON	125 562 octets	Oui, pour ce guide
Point de reprise corpus	371 034 octets	Pour reprendre l'entraînement, pas nécessaire à la prédiction
Expérience interactive récente complète	1 428 396 octets	Pour reprendre l'historique interactif et réemployer ses recettes
Pilote JSONL, 48 cas et 2 stratégies	666 890 octets	Pour audit ou nouvel entraînement

Une extrapolation proportionnelle du pilote donne **13,89 Go de JSONL pour un million de cas à deux stratégies**, hors maillages et sauvegardes. Quatre stratégies augmentent cette quantité ; elles ne doublent pas exactement le total puisque certaines métadonnées sont communes. Ce chiffre repose sur un petit échantillon et doit être recalibré sur un lot plus représentatif. À 1 Mo par maillage conservé, quatre millions d'essais ajouteraient 4 To ; à 10 Mo, 40 To. Ces exemples expliquent pourquoi il faut garder les journaux utiles et conserver les maillages de manière sélective.

Les poids ne grossissent pas proportionnellement au nombre de configurations tant que l'architecture reste fixe. Un modèle entraîné sur beaucoup plus de cas peut rester de l'ordre de centaines de ko. Ce sont les expériences, les journaux, les versions, les copies de sources et les exports qui augmentent.

Consulter `TAILLES_ET_CAPACITES.md` pour l'inventaire détaillé et `MESURES_M3.md` pour la taille réellement installée. `scripts/inventory_sizes.py` reproduit les mesures en séparant sources, preuves, dépendances et outils. La taille des fichiers n'est pas la RAM consommée pendant un maillage.

Envoyer le modèle à une autre personne

Le destinataire a besoin du code compatible et de ses dépendances, puis du fichier de poids correspondant au bon rôle. Un JSON de poids ne constitue pas une application autonome. Le binaire Rust est propre à la plateforme ; les fichiers JSON de poids sont portables entre plateformes compatibles.

Le guide corpus se place dans `DOSSIER_ATELIER/models/corpus.json` pour le parcours `learn`, dont l'option `-o DOSSIER_ATELIER` définit l'espace de travail. Le guide historique s'utilise en CLI avec `mesh --advisor chemin/guide.json` ; le pont Python conserve sa copie dans `DOSSIER_ATELIER/models/guide.json`. Sa présence seule ne demande pas son utilisation à une commande `mesh` sans `--advisor`. Ces fichiers ont des schémas différents : ne pas les interchanger, ni modifier artificiellement leur indicateur d'activation.

Le guide interactif appartient au dossier mémoire, sous `DOSSIER_MEMOIRE/model.json`, sélectionné avec `learn --memory DOSSIER_MEMOIRE`. **Copier uniquement ce JSON dans une mémoire neuve ne permet pas de poursuivre son apprentissage.** La mise à jour cumulative réclame les observations des groupes déjà appris et signale un historique incomplet si elles manquent. Pour poursuivre ce guide ou réemployer ses recettes, transférer un dossier `experience/` cohérent, comprenant index, journal, sources et modèles, avec les calculs arrêtés et après

examen de son contenu. Le modèle seul reste transportable pour examiner ses poids et ses métadonnées ; il ne constitue pas un import autonome de mémoire.

`scripts/share_model.py` produit un ZIP léger avec modèle, manifeste d'intégrité et instructions, sans copier automatiquement de corpus, dossier mémoire ou fichier CAO séparé. Pour le guide historique, il retire aussi le champ `training_configs`, qui peut embarquer des géométries et n'est pas requis par le chargeur de prédiction ; les poids, les preuves et l'état d'activation sont conservés. Les paquets de démonstration de `output/partage/` occupent environ 42 ko pour le corpus, 19 ko pour l'interactif et 4 ko pour l'historique. Ils restent désactivés comme les modèles sources. Conserver un jeu d'exemples publics de validation facilite la vérification par le destinataire.

7. Installer et commencer

Pour une installation neuve, utiliser Python 3.11, 3.12 ou 3.13 et Cargo si aucun noyau de la plateforme n'est fourni. Les commandes suivantes sont à lancer depuis `atelier_mailleur/`. Le premier téléchargement nécessite Internet ; les calculs fonctionnent ensuite localement.

```
python3.12 scripts/install.py
python3.12 scripts/launch.py
```

Sur ce Mac déjà préparé, le lanceur utilise l'environnement local :

```
sh Lancer.command
.venv/bin/python -m atelier doctor
.venv/bin/python -m atelier mesh examples/atelier_plate.json \
  -o output/premier_maillage --formats msh,inp
```

Pour apprendre sur une pièce en gardant une mémoire commune :

```
.venv/bin/python -m atelier learn examples/apprentissage_plaque_zones.json \
  -o output/mon_atelier --max-trials 4 --epochs 20 \
  --max-seconds 60 --formats msh
.venv/bin/python -m atelier memory --workspace output/mon_atelier
```

Pour une CAO importée, utiliser un projet dont `geometry.kind` vaut `import` et fournir `--source piece.step`. Un projet portable `.amp` contient déjà sa source. Toujours contrôler les unités d'entrée, le mode et la qualité avant utilisation dans un solveur.

Pour constituer un pilote équilibré puis l'entraîner :

```
.venv/bin/python -m atelier corpus plan --directory output/pilote \
  --workspace output/mon_atelier --structured 24 --random 24 --strategies 2
.venv/bin/python -m atelier corpus run --directory output/pilote \
  --workspace output/mon_atelier --limit 48 --max-seconds 120
.venv/bin/python -m atelier corpus train --directory output/pilote \
  --workspace output/mon_atelier --epochs 3 --batch-size 16 \
  --max-records 0 --max-seconds 60
.venv/bin/python -m atelier corpus status --directory output/pilote \
  --workspace output/mon_atelier
```

Pour la campagne cible, créer un autre dossier avec les quotas 250000 et 750000. Relancer `corpus run` traite le lot suivant. Le logiciel ne lance pas automatiquement toute la campagne. Le corpus se planifie aussi depuis Qt. Le guide est copié dans le dossier de l'atelier après entraînement ; sa présence ne prouve pas son activation.

8. Fonctionnement de l'API

Deux interfaces de programmation coexistent : la bibliothèque Python `NativeSession` et l'API HTTP locale. Le pont interne vers Rust utilise un fichier JSON par requête et un nouveau processus `mailleur-core --request FICHIER`. Cette version n'utilise pas un serveur Rust permanent en JSON-lines.

```
.venv/bin/python -m atelier serve --host 127.0.0.1 --port 8765 \
  --workspace output/api --workers 1 --timeout 300 --no-browser
```

Le client lit `GET /api/info` pour connaître le moteur, les formats et le jeton de session. Il envoie ensuite `POST /api/jobs` avec le projet JSON et l'en-tête `X-Atelier-Token`. Le serveur répond immédiatement avec un identifiant et l'état `queued` ; un processus de travail lance Rust et les exports. Le client suit `GET /api/jobs/{id}` jusqu'à `completed`, `failed` ou `cancelled`, puis récupère `/bundle` ou `/download?format=msh`. Le rapport est disponible à `/report`.

`POST /api/uploads` reçoit une CAO encodée en base64 et renvoie `upload_id`. Le client joint cet identifiant au projet importé. La limite de source est 50 Mio ; base64 augmente le volume transféré d'environ un tiers. Cette entrée est conservée en mémoire pendant le décodage ; l'optimisation par blocs porte sur les sorties téléchargées.

`POST /api/learning` recherche et mémorise une recette sur la pièce ; `/api/training` lance le conseiller historique. Ces routes ne sont pas l'entraîneur du grand corpus. Le corpus reste accessible en CLI, Python et interface Qt. La file accepte 16 travaux en attente ou actifs ; le nombre de travailleurs est réglable de 1 à 8. Le délai du serveur couvre le traitement et les exports.

Ce serveur est prévu pour l'ordinateur local. Son jeton protège les mutations et les contrôles d'hôte/origine protègent le contexte navigateur ; le jeton est accessible via `/api/info` aux clients qui peuvent joindre le service. Il ne constitue pas une authentification multi-utilisateur pour un service public. Pour des personnes distantes, le partage du projet et du modèle permet à chacun de lancer son instance locale. Un hébergement partagé demanderait une conception dédiée de l'authentification, des espaces utilisateurs, des quotas et du déploiement.

Le document `API_UTILISATION.md` donne toutes les routes, un client Python complet et les commandes cURL. Le fonctionnement par fichiers facilite le diagnostic : un dossier de travail conserve requête, rapport, journal, maillage et exports. Un échec du processus retourne un diagnostic ; au redémarrage les travaux interrompus sont marqués en échec, tandis que la reprise du corpus suit ses points de reprise propres.

9. Optimisations appliquées et améliorations suivantes

La rc5 initiale contenait déjà des optimisations d'entraînement, de reprise, de qualité et de rendu. Leurs mesures Linux sont dans `OPTIMISATIONS_RC5.md`. Elles ne doivent pas être présentées comme des résultats mesurés sur Apple M3.

Cette révision réduit les dépendances Qt aux modules utilisés, transfère les gros téléchargements HTTP par blocs, évite le chargement des aperçus sur les routes de fichiers et de rapports, et vérifie l'exécution du noyau lors de l'installation. Les dépendances Cargo restent verrouillées. L'apprentissage interactif borne chaque vecteur d'entrée une fois et le réutilise, avec comparaison numérique bit à bit contre le calcul initial. Les gains retenus et leurs limites sont détaillés dans `MESURES_M3.md`.

Pour améliorer la qualité de l'apprentissage, la priorité est de mesurer le gain de sélection sur des familles et des pièces réservées, puis d'augmenter la diversité utile. Un seuil d'activation basé sur la validation est plus informatif que le seul nombre d'époques. Les géométries réelles nouvelles et les cas difficiles doivent être examinés séparément des variantes très proches.

Pour accélérer davantage, profiler d'abord la génération, l'inspection, les contrôles, les écritures et les exports sur une charge représentative. Si la CAO domine, accélérer les quelques milliers de poids aura peu d'effet sur le temps total. Un cache persistant doit intégrer source, unités, critères et versions numériques ; une ancienne réussite doit être revalidée. Le parallélisme exige des dossiers et mémoires cohérents par travail, des budgets et des mesures de RAM. Ajouter un GPU aux petits réseaux ne garantit pas un progrès.

Le tout-hexa, une représentation GNN, une adaptation pilotée par le solveur ou un service HTTP multi-utilisateur sont des évolutions architecturales. Elles demandent des objectifs et des validations distincts ; elles ne sont pas des options déjà disponibles dans cette livraison.

10. Préparer GitHub et contribuer

Publier les sources, README, documentation, exemples publics, tests, verrou Cargo et workflows CI. Le `.gitignore` fourni exclut les environnements, la chaîne de compilation et les dossiers de travail usuels. Il ne peut pas deviner toutes les CAO privées : examiner les fichiers retenus avant tout envoi. Le projet n'a pas été publié sur GitHub par cette intervention.

Distribuer les poids dans un ZIP de modèle ou une release, avec architecture, schéma, version, provenance des données, métriques et état d'activation. Garder les gros corpus et leurs copies hors de l'historique Git courant ; un fichier de poids n'a pas besoin de transporter plusieurs gigaoctets de journaux. Les preuves de livraison servent de références techniques, sans être un corpus de production.

Le code propre au projet n'a pas encore de licence de publication choisie par son titulaire. Le fichier `THIRD_PARTY.md` répertorie les composants tiers et doit accompagner la diffusion. Gmsh est distribué sous GPL avec ses conditions propres ; Qt/PySide dispose également de conditions spécifiques. Choisir une licence pour le code personnel ne remplace pas les notices des dépendances. Aucune licence ni identité d'auteur n'a été inventée dans cette révision.

Un contributeur peut partir des commandes suivantes, après installation :

```
cargo test --locked --release --manifest-path rust_core/Cargo.toml
QT_QPA_PLATFORM=offscreen .venv/bin/python -m unittest discover -s tests -v
.venv/bin/python scripts/benchmark_local.py
```

Sur ce Mac, la chaîne Rust isolée demande les variables `CARGO_HOME` et `RUSTUP_HOME` pointant vers `.toolchain/cargo` et `.toolchain/rustup`. Pour une installation normale de Rust, ces variables spécifiques ne sont pas nécessaires. Le script d'empaquetage réutilise le binaire compilé localement et crée un manifeste SHA-256 ; l'archive produite n'embarque pas les environnements de développement.

Comparer une modification sur les mêmes entrées, graines, critères et versions de bibliothèques. Séparer temps de calcul, qualité, taux d'échec, RAM, écritures et conversions. Une optimisation qui change les nœuds exige une étude de qualité ; une optimisation supposée identique doit comparer ses résultats. La CI proposée pour trois systèmes reste distincte des tests réellement exécutés localement et des essais à mener dans Cast3M, Abaqus ou ANSYS.

11. Petit glossaire

Terme	Sens dans ce projet
CAO	Géométrie source, avec surfaces, arêtes, volumes et topologie
Maillage	Nœuds et connectivité des éléments utilisés par un calcul
Jacobienne	Indicateur lié à l'orientation et à la transformation d'un élément
minSICN	Indicateur de forme normalisé utilisé dans le contrôle de qualité
Recette	Ensemble de paramètres de maillage à exécuter et contrôler
Corpus	Configurations et résultats d'essais servant à l'apprentissage
Époque	Passage d'entraînement sur les exemples effectivement disponibles
Mini-lot	Groupe borné d'exemples utilisé pour une mise à jour des poids
Checkpoint	État permettant de reprendre un entraînement compatible
Inférence	Calcul d'une prédiction à partir de poids déjà entraînés
API	Contrat permettant à un autre programme de demander des opérations
CLI	Interface par commandes dans un terminal

12. Références et preuves

Le code de `atelier/` et `rust_core/src/` fait foi pour cette version. Les documents `ARCHITECTURE.md`, `CORPUS_ET_LANGAGES.md`, `APPRENTISSAGE_CONTINU.md`, `FORMATS.md` et `LIMITES.md` développent les contrats spécialisés. `TAILLES_ET_CAPACITES.md`, `API_UTILISATION.md` et `MESURES_M3.md` complètent ce guide.

Les preuves initiales sont dans `verification/native_release/`, `verification/corpus/`, `verification/continuous_learning/` et `verification/optimization/`. Les nouvelles preuves sont dans `verification/local_m3/`. Les deux provenances restent séparées.

Références officielles consultées pour les dépendances : Gmsh et son API, modules des paquets Qt for Python, PySide6-Essentials, gestion de Python par uv. Les tailles installées proviennent de mesures locales, pas des tailles annoncées sur ces pages.

Annexe A — Mesures sur Apple M3

Machine et méthode

Mesures du 4 novembre 2026 sur le MacBook Pro de l'utilisateur : **Apple M3, 8 cœurs CPU, GPU 10 cœurs, mémoire unifiée 16 Go, macOS 27.0, arm64**. L'espace disponible relevé au début était d'environ 797 Gio. Aucun numéro de série matériel n'est inclus dans les livrables.

Environnement isolé : Python **3.12.16**, Rust/Cargo **1.99.1**, Gmsh **4.15.2**, meshio **5.3.5**, h5py **3.16.0**, NumPy **2.5.4**, PySide6-Essentials et shiboken6 **6.12.0**. Les versions sont consignées dans `verification/local_m3/requirements-resolved.txt`. Les dépendances Cargo utilisent `Cargo.lock` et le profil release. Le GPU n'est pas utilisé par les réseaux actuels.

Les mesures de génération utilisent un seul thread demandé au mailleur, sans conseiller neuronal. Trois répétitions sont conservées ; la valeur annoncée est leur médiane. Le temps comprend la préparation de la requête, le démarrage du noyau Rust, le calcul, les fichiers et le chargement des rapports/aperçus par `NativeSession`. Les exports sont mesurés séparément. Ce protocole ne mesure pas l'ouverture de la fenêtre, un transfert HTTP ni un calcul mécanique dans Cast3M.

Génération de maillages

Cas	Nœuds	Éléments du domaine	Génération, médiane	Export MSH, médiane
Plaque percée à congés, exemple 2D	363	624 triangles	0,099 s	0,159 s
Plaque 3D percée	1 696	5 802 tétraèdres	0,304 s	0,191 s
Coque sphérique	382	760 triangles	0,171 s	0,158 s
Boîte 20×16×8 mm, maille cible 0,65 mm	9 156	44 895 tétraèdres	0,668 s	Non mesuré

Les éléments de bord sont en plus des éléments du domaine indiqués. Les rapports des quatre cas sont admissibles selon leur politique : aucun élément invalide détecté. Les exemples utilisent leurs réglages livrés ; la boîte emploie Delaunay, ordre 1, sans zone critique, avec un minimum de qualité exigé de 0,05. Ce n'est pas le même cas que l'ancienne boîte Linux avec douze zones exigeant des seuils élevés : on ne peut pas attribuer leur différence de temps à la seule machine ou à la nouvelle optimisation.

Pour la boîte dense, le plus haut RSS des processus Rust lancés par le processus de mesure est **123 322 368 octets**, soit **117,61 Mio**. Le pic propre au processus Python est **75 005 952 octets**, soit **71,53 Mio**. Ces pics sont séparés et ne décrivent pas une mesure simultanée de la mémoire de toute l'application Qt. Les autres cas n'ont pas de mesure RSS dédiée. Une CAO complexe peut exiger beaucoup plus de mémoire, indépendamment du nombre final d'éléments.

Les petits exemples sont donc utilisables immédiatement sur ce Mac. Les temps ne garantissent aucune vitesse sur des volumes industriels, des défauts complexes, des couches très fines ou des maillages de millions d'éléments. Les recherches à plusieurs recettes et les exports multiples ajoutent leurs coûts.

Sources : `verification/local_m3/meshing.json` et `dense.json`. Reproduction des trois exemples :

```
.venv/bin/python scripts/benchmark_local.py --repetitions 3
```

Un cycle réel de corpus et d'entraînement

Un nouveau pilote a été produit sur le Mac, sans réutiliser les lignes des preuves Linux : 24 cas structurés et 24 aléatoires, deux stratégies par cas, 24 familles. Deux lots de 24 ont été exécutés dans deux objets `NativeSession` successifs, pour contrôler la reprise.

Phase ou résultat	Observation
Planification	0,021 s ; aucun maillage à cette étape
Premier lot de 24 cas	3,737 s
Deuxième lot repris, 24 cas	4,735 s
Total production des 48 cas	8,472 s
Inspections et évaluations backend	140 appels, dont les inspections
Cas ayant produit un maillage	46 sur 48
Cas ayant une stratégie admissible	45 sur 48
Stratégies	96 résultats : 88 admissibles et 8 refusés
JSONL produit	666 857 octets, sans conservation de tous les maillages
Entraînement corpus, 3 époques, mini-lots de 16	0,810 s, pont et publication compris
Réseau	128→32→1, 4 161 paramètres
Répartition des cas	32 apprentissage, 8 validation, 8 test
Poids `current.json`	125 518 octets
Checkpoint de reprise	370 945 octets
Décision d'activation	`deploy_enabled=false`

Deux cas sont des géométries invalides attendues. Ils sont enregistrés comme refus, sans maillage accepté artificiellement. Le réseau a été entraîné et sauvegardé ; la récompense de sélection en validation vaut **0,75212659**, identique à celle de la référence. En test, elle vaut **0,84445310**, également identique à la référence. L'activation reste donc refusée. Ce pilote confirme le fonctionnement de la chaîne sur le Mac ; il ne démontre pas que l'IA améliore déjà les maillages.

Le ZIP `output/partage/Modele_corpus_M3_pilote.zip` contient ces poids, un manifeste et les instructions, pour **42 015 octets**. Il conserve la décision de non-activation. Aucun million de configurations n'a été généré ni entraîné pendant cette intervention. Les 0,810 seconde ne sont pas une estimation d'entraînement d'un million de cas.

Une projection purement linéaire des 8,472 secondes sur un million de cas donnerait environ **49 heures à deux stratégies**, sur des configurations comparables à ce minuscule pilote. Cette arithmétique ignore les effets d'échelle, les géométries plus difficiles, les contrôles et les changements de distribution ; elle ne constitue pas une prévision de campagne. Mesurer d'abord un lot plus représentatif, puis plusieurs segments, permet de construire un budget crédible. Quatre stratégies changent le coût et le volume des journaux.

Source : `verification/local_m3/pilot.json`. Les commandes de reproduction du pilote figurent dans le guide complet.

Optimisations nouvelles vérifiées

Réseau interactif

La version initiale bornait chaque caractéristique à chaque neurone, puis à nouveau pour le gradient. La révision prépare une fois le vecteur borné et le réutilise. Le test Rust compare l'ancien calcul et le nouveau sur plusieurs graines et valeurs limites ; les poids et prédictions sont identiques bit à bit.

Le benchmark isolé utilise 8 192 exemples synthétiques et 16 passes, cinq répétitions, avec la même chaîne de compilation pour les deux sources. Il a été relancé après la suite de tests.

Mesure MLP seul, entrées dans [0,1]	rc5 initiale	Révision	Rapport de vitesse
Inférence	0,121729 s	0,108129 s	1,13×
Gradients / SGD	0,201809 s	0,161080 s	1,25×

Sur la fixture avec entrées saturées, les rapports valent environ 1,12× et 1,26×. La génération CAO, les maillages, la préparation de l'historique et la publication des modèles sont exclus de ce benchmark. Le gain du noyau SGD n'est donc pas un gain global de 25 % sur l'application ou les maillages.

Source retenue : `verification/local_m3/live_inputs_confirmed.json`. La première série `live_inputs.json` est conservée comme mesure exploratoire ; ses médianes ne sont pas utilisées pour les chiffres ci-dessus.

Téléchargements de l'API

Les fichiers sont envoyés par blocs de 1 Mio ; le serveur évite de charger l'aperçu sur les routes de téléchargement et de rapport. Un export contenant plusieurs fichiers est assemblé dans un ZIP temporaire sur disque. Le résultat téléchargé reste identique et les tests vérifient que les gros fichiers ne sont pas lus entièrement par `read_bytes`.

Mesure isolée sous Python 3.12, fichier creux de 128 Mio envoyé vers `/dev/null`, dans deux processus avec les mêmes imports : pic RSS de **161 857 536 octets (154,36 Mio)** pour l'ancienne lecture complète, contre **29 655 040 octets (28,28 Mio)** pour le transfert par blocs. Environ **126,08 Mio de pic sont évités** sur ce scénario. Ce n'est pas une mesure de débit réseau, ni de génération. L'upload base64 reste intégralement décodé en mémoire.

Source : `verification/local_m3/download_memory.json`. Reproduction :

```
.venv/bin/python scripts/benchmark_download_memory.py
```

Installation allégée et portable

L'application importe QtCore, QtGui et QtWidgets ; le paquet **PySide6-Essentials** suffit à ces usages. Le projet demandait auparavant le méta-paquet PySide6, dont la résolution 6.12 sur ce Mac téléchargeait aussi WebEngine, PDF et Addons, inutilisés par l'application. Les tailles annoncées par le résolveur pour ces trois roues étaient respectivement environ 238,2, 8,2 et 86,4 Mio, soit **environ 333 Mio de téléchargements évités** pour cette résolution précise. Ce chiffre est une taille de téléchargement, pas une mesure de l'économie décompressée sur tous les systèmes.

Les 163 tests Python, dont les tests Qt exécutés en mode offscreen, passent avec Essentials seul. Les composants inutiles ne sont pas installés. Les téléchargements et caches temporaires de préparation ont été nettoyés dans le seul dossier du projet. L'installateur contrôle désormais l'exécution du noyau avec `--version`, suit la priorité du pont natif et compile Cargo avec `--locked`. Le lanceur contrôle aussi les bibliothèques nécessaires et borne ses vérifications par un délai.

La documentation officielle des paquets Qt confirme la séparation des modules : Qt for Python, Package Details, PySide6-Essentials.

Place occupée après préparation

Mesure des longueurs logiques de fichiers, sans suivre les liens symboliques, après nettoyage du cache de téléchargement. Les tailles peuvent évoluer après de nouveaux calculs ou une compilation.

Dossier	Taille observée	Nécessité et diffusion
`.venv/`	509,70 Mo	Dépendances d'exécution ; recréer chez le destinataire
`.runtime/`	70,67 Mo	Python 3.12 local ; nécessaire à cet environnement précis
`.bootstrap/`	36,93 Mo	Outil de préparation uv ; inutile au calcul courant
`.toolchain/`	493,71 Mo	Rust et cache Cargo ; utile pour développer/recompiler
`rust_core/target/`	64,45 Mo	Compilation et tests ; seul le binaire release est nécessaire au calcul
`output/`	41,50 Mo lors du relevé	Maillages de mesure et pilote ; augmente à l'usage
Sources, exemples, preuves et documentation	Environ 75 Mo avant les documents bureautiques finaux	Livrable partageable, détails dans l'inventaire

L'ensemble local de développement est d'environ **1,29 Go** à ce relevé, contre environ **0,66 Go** pour les sources/preuves, l'environnement Python, le Python local et le noyau nécessaires à l'exécution, hors nouveaux résultats. Le destinataire reçoit une archive source bien plus petite : il n'a pas à recevoir les 1,29 Go du poste de développement. Les environnements `.venv` ne sont pas des installations portables à recopier entre machines.

Les fichiers de poids représentent quelques dizaines ou centaines de ko. L'entraînement d'un corpus plus grand ne les fait pas croître proportionnellement si l'architecture reste identique. Le stockage du corpus, les sauvegardes et les maillages conservés dominent la croissance. L'inventaire exact actualisé est dans `verification/local_m3/inventory.json` ; le manifeste d'archive décrit le contenu exact du ZIP final.

Validations et utilisation immédiate

163 tests Python réussis en 29,300 s, aucun ignoré ; 111 tests Rust réussis en 4,79 s, aucun ignoré ni échec. Les temps Rust excluent la compilation de la suite. Le binaire release a été compilé pour Apple M3 avec les dépendances verrouillées. Les avertissements de code Rust inutilisé ne sont pas des échecs de tests.

Le client de la documentation HTTP a aussi été exécuté sur un serveur local temporaire pour une plaque et pour le STEP fourni à deux volumes : les calculs et téléchargements ZIP aboutissent. Les rapports et conversions sont contrôlés par les tests ; aucun nouveau calcul de solveur Cast3M/Abaqus/ANSYS n'a été réalisé ici. Windows, Linux avec cette révision locale et les autres configurations macOS nécessitent leurs propres vérifications.

Sur ce Mac, depuis le dossier `atelier_mailleur/` :

```
sh Lancer.command
.venv/bin/python -m atelier doctor
```

Le lanceur peut être exécuté avec le Python système car l'environnement `.venv` compatible existe déjà. Pour recompiler avec la chaîne isolée :

```
export CARGO_HOME="$PWD/.toolchain/cargo"
export RUSTUP_HOME="$PWD/.toolchain/rustup"
export PATH="$CARGO_HOME/bin:$PATH"
cargo build --locked --release --manifest-path rust_core/Cargo.toml
```

Le ZIP final inclut le noyau Apple M3 de cette révision. Par défaut, l'empaquetage exclut les binaires des autres plateformes qui n'ont pas été recompilés ici, afin qu'un destinataire reconstruise les sources modifiées avec Cargo. Il n'inclut ni `.venv`, ni Python local, ni Rust, ni les caches. Le paquet n'est pas une application macOS signée/notariée.

Annexe B — Utiliser l'API

Ce guide décrit le code présent dans ce dépôt, issu de la version rc5. Trois moyens d'intégration existent : le serveur HTTP local, la classe Python `NativeSession`, et le programme Rust `mailleur-core --request`. Ils utilisent le même moteur. L'interface Qt utilise surtout `NativeSession` ; le serveur HTTP fournit aussi une petite interface web.

Cette version n'utilise pas le serveur JSON-lines à 52 ordres décrit dans l'ancienne synthèse Mailleur v2.5. Les noms de commandes et les formats de modèles de ces deux projets ne sont pas interchangeables.

1. Démarrer et comprendre le service

Depuis la racine du dépôt, après installation :

```
.venv/bin/python -m atelier doctor
.venv/bin/python -m atelier serve --host 127.0.0.1 --port 8765 \
  --workspace ./output_api/atelier --workers 1 --timeout 300 --no-browser
```

Sous Windows, remplacer `.venv/bin/python` par `.venv\Scripts\python.exe`. Garder cette console ouverte et exécuter les exemples clients dans une seconde console. `Ctrl+C` arrête le service et annule ses calculs actifs. Sans `--no-browser`, la commande ouvre l'interface web.

Le service reçoit un projet JSON et éventuellement un fichier CAO. Il valide les paramètres, crée un identifiant de calcul, puis lance un processus Python isolé. Ce processus écrit une requête JSON pour Rust et lance le noyau natif. Rust pilote Gmsh/OpenCASCADE pour la géométrie et le maillage, contrôle la qualité et effectue l'apprentissage. Les conversions de fichiers passent aussi par les adaptateurs Python meshio/h5py. Le client interroge l'état du calcul puis télécharge les résultats.

```
sequenceDiagram
    participant C as Client HTTP
    participant S as Serveur Python
    participant W as Processus de calcul Python
    participant R as Noyau Rust et Gmsh
    C->>S: GET /api/info
    S-->>C: Jeton et capacités
    C->>S: POST /api/jobs + projet + jeton
    S-->>C: 202 + identifiant
    S->>W: request.json
    W->>R: mailleur-core --request native_request.json
    R-->>W: mesh.msh, report.json, preview.json
    W-->>S: Exports et result.json
    C->>S: GET /api/jobs/identifiant
    S-->>C: État, rapport et aperçu
    C->>S: GET /api/jobs/identifiant/bundle
    S-->>C: ZIP des exports
```

Un maillage classique fonctionne sans modèle entraîné. Le guide et la mémoire proposent des réglages ; ils ne remplacent pas le générateur géométrique et les contrôles de qualité.

2. Accès local, authentification et limites

- L'adresse par défaut est `http://127.0.0.1:8765`. Le service n'emploie ni TLS, ni comptes utilisateurs, ni gestion de droits par personne. Le conserver sur la boucle locale pour cet usage. `--host 0.0.0.0` l'expose au réseau et ne constitue pas une configuration de service public.
- `GET /api/info` fournit `token`. Chaque `POST` doit envoyer ce jeton dans l'en-tête `X-Atelier-Token`. Le jeton change au redémarrage. Lorsqu'un navigateur envoie `Origin`, son hôte et son port doivent correspondre à l'en-tête `Host` de la requête. Aucune configuration CORS pour un site tiers n'est fournie.
- Les routes `GET`, y compris les résultats et le jeton, ne demandent pas de jeton. Celui-ci protège les mutations locales ; il ne fournit pas une isolation entre utilisateurs de l'ordinateur ou du réseau.
- Un fichier CAO importé doit contenir de 1 octet à **50 Mio**. Il est envoyé en base64 dans JSON : le corps transmis occupe environ un tiers de plus. Une archive `.amp` a une tolérance supplémentaire de 1 Mio au décodage, puis subit les contrôles du lecteur de projets.

- Les extensions d'import admises sont `.step`, `.stp`, `.iges`, `.igs`, `.brep`, `.stl`, `.msh`, `.vtk`, `.unv`. Les scripts CAO exécutables ne sont pas admis. MSH/VTK/UNV servent à importer une peau triangulée ; ils ne garantissent pas la récupération d'une CAO analytique.
- Le serveur accepte **1 à 8 calculs simultanés**, selon `--workers`, et au plus **16 calculs en attente ou actifs**. Un seul entraînement historique peut être en attente ou actif à la fois. Sur un Mac de 16 Go, commencer avec un worker et un fil de maillage.
- `--timeout` vaut 300 secondes par défaut, avec une plage de 1 à 86 400 secondes. Il couvre l'ensemble du processus de calcul, dont les conversions d'export. Le budget `search.max_seconds` d'un apprentissage ne relève pas cette limite globale.
- `mesh.max_elements` est limité à 2 000 000, `mesh.threads` à 32, et l'ordre des éléments à 1 ou 2. Un plafond d'éléments ne garantit pas un plafond mémoire strict pendant les opérations internes du mailleur.
- Les téléchargements existants sont transférés par blocs de 1 Mio. Un export composé de plusieurs fichiers est assemblé dans un ZIP temporaire sur disque. Les téléchargements et rapports évitent de charger l'aperçu géométrique. Les requêtes d'import JSON/base64 et la réponse complète d'un calcul conservent, elles, un coût mémoire lié à leur taille.

Le workspace contient `jobs/`, `uploads/`, `models/`, `experience/`, puis les sessions `native/`. L'historique n'est pas purgé automatiquement. Les maillages, sources CAO et expériences restent des fichiers locaux ; prévoir leur nettoyage ou leur archivage. Les identifiants d'import n'ont de sens que dans le workspace du serveur concerné.

3. Routes HTTP disponibles

Tous les corps de requête ci-dessous sont des objets JSON. Envoyer `Content-Type: application/json` pour les `POST`. Les réponses sont JSON, sauf les téléchargements explicitement indiqués.

Méthode et route	Corps ou paramètre	Résultat
`GET /api/info`	Aucun	Version, jeton, état du backend, formats, exemples, plafond d'import, nombre de workers, disponibilité du guide et liste des 30 derniers calculs
`GET /api/memory`	Aucun	Compteurs et état de la mémoire d'expériences, lus par Rust
`POST /api/uploads`	`filename`, `content_base64`	**201** : `id`, `filename`, `bytes` ; conserver `id` comme `upload_id`
`POST /api/jobs`	`project`, `upload_id` facultatif, `use_advisor` facultatif, booléen	**202** : calcul de maillage créé
`POST /api/learning`	`project`, `upload_id` facultatif, `search` facultatif	**202** : recherche de réglages et apprentissage sur cette géométrie
`POST /api/training`	`{}`	**202** : entraînement du guide historique sur les exemples de démonstration ; cette route ne lance pas le corpus d'un million de configurations
`GET /api/jobs/{id}`	Identifiant retourné à la création	État du calcul ; une fois terminé, rapport, aperçu `preview` et liste des exports `files`
`POST /api/jobs/{id}/cancel`	`{}`	Demande d'annulation et état courant
`GET /api/jobs/{id}/report`	Calcul terminé	Rapport JSON téléchargé sous le nom `rapport_qualite.json`
`GET /api/jobs/{id}/bundle`	Calcul terminé avec exports	ZIP des exports, du projet et des rapports ; pas un paquet de modèle entraîné

Méthode et route	Corps ou paramètre	Résultat
`GET /api/jobs/{id}/download ?format=msh`	Format disponible dans `files`	Fichier demandé ; ZIP si le format requiert plusieurs fichiers
`POST /api/projects/save`	`project`, `upload_id` facultatif	Archive portable `.amp`, de type ZIP, contenant le projet et sa source lorsqu'elle est jointe
`POST /api/projects/open`	`content_base64` d'une `.amp`	Projet normalisé, nouvel `upload_id` si source présente et son nom `filename`

Les formats d'export publics sont `msh`, `med`, `inp`, `vtu`, `unv`, `ansys`, `cast3m`. Le champ `formats` de `/api/info` fournit les extensions et limites. ANSYS produit un script APDL de maillage MESH200. Cast3M produit un script d'import Gibiane accompagné d'UNV. Les exports ne définissent ni matériau, ni chargement, ni conditions aux limites d'un calcul mécanique.

Les routes `bundle`, `download` et `report` exigent un calcul `completed`. Un entraînement ne dispose pas de maillage et ne fournit donc pas de paquet d'exports. Le serveur HTTP ne publie pas de routes pour le corpus, l'examen CAO, le remaillage local ou le transfert des modèles : utiliser la CLI ou `NativeSession` pour ces opérations.

Structure d'un projet

Un projet minimal, exprimé en millimètres :

```
{
  "schema_version": 1,
  "name": "Plaque API",
  "mode": "planar",
  "geometry": {"kind": "plate", "width": 2, "height": 1},
  "mesh": {
    "size": 0.3,
    "min_size": 0.1,
    "threads": 1,
    "max_elements": 5000
  },
  "units": {"input": "mm", "internal": "m"}
}
```

Les modes sont `planar`, `surface`, `volume`. Les géométries sont `plate`, `polygon`, `box`, `sphere`, `bracket`, `import`. Les exemples complets sont dans `examples/atelier_*.json`. La normalisation et ses contraintes figurent dans `atelier/projects.py`. Les unités d'entrée et d'export sont `mm`, `cm`, `m`, `in` ; le moteur travaille en mètres. Les unités des dimensions CAO importées doivent correspondre au réglage du projet.

Pour une CAO importée, mettre `"geometry": {"kind": "import"}` et joindre `upload_id` au niveau de la requête HTTP. Ne pas mettre de chemin, de nom de fichier ou de script dans `geometry`.

Réglages d'apprentissage

```
{
  "project": {"name": "Exemple", "geometry": {"kind": "plate", "width": 2, "height": 1}},
  "search": {
    "max_trials": 4,
    "epochs": 20,
    "seed": 42,
    "max_seconds": 30,
    "use_memory": true
  }
}
```

`max_trials` accepte 1 à 64, `epochs` 1 à 200, `max_seconds` 1 à 86 400, `seed` un entier de 0 à 4 294 967 295, et `use_memory` un booléen. Les valeurs par défaut sont respectivement 12, 100, 240, 42 et `true`. Les réglages `learning` du projet peuvent fournir les valeurs par défaut hors graine ; `search` les remplace. Les noms inconnus dans `search` sont refusés.

Les zones critiques, la politique de qualité et les régions de raffinement sont des champs du projet, illustrés dans `examples/apprentissage_plaque_zones.json` et `examples/apprentissage_volume_zone.json`. La géométrie seule ne fournit pas une distribution de contraintes mécaniques : désigner les zones sensibles ou apporter les informations d'un calcul adapté.

4. Exemple cURL depuis la racine du dépôt

Ces commandes utilisent un shell macOS/Linux et le Python de l'environnement local. Le serveur doit déjà être démarré.

```
mkdir -p output_api/client
MAILLEUR_URL=http://127.0.0.1:8765
MAILLEUR_PY=.venv/bin/python

curl --fail-with-body -sS "$MAILLEUR_URL/api/info" -o output_api/client/info.json
MAILLEUR_TOKEN="$("$MAILLEUR_PY" -c 'import json; print(json.load(open("output_api/client/info.json"))["token"])')"

"$MAILLEUR_PY" - <<'PY'
import json
from pathlib import Path
project = json.loads(Path("examples/atelier_plate.json").read_text(encoding="utf-8"))
Path("output_api/client/request.json").write_text(
    json.dumps({"project": project, "use_advisor": False}), encoding="utf-8")
PY

curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs" \
  -H 'Content-Type: application/json' -H "X-Atelier-Token: $MAILLEUR_TOKEN" \
  --data-binary @output_api/client/request.json -o output_api/client/created.json
MAILLEUR_JOB="$("$MAILLEUR_PY" -c 'import json; print(json.load(open("output_api/client/created.json"))["id"])')"

curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs/$MAILLEUR_JOB"
```

Répéter la dernière commande jusqu'à `completed`, `failed` ou `cancelled`. Le client Python de la section suivante automatise cette attente. Après `completed` :

```
curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs/$MAILLEUR_JOB/report" \
  -o output_api/client/rapport.json
curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs/$MAILLEUR_JOB/bundle" \
  -o output_api/client/maillage_exports.zip
curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs/$MAILLEUR_JOB/download?format=msh" \
  -o output_api/client/mesh.msh
```

Pour annuler un calcul encore en attente ou actif :

```
curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs/$MAILLEUR_JOB/cancel" \
  -H 'Content-Type: application/json' -H "X-Atelier-Token: $MAILLEUR_TOKEN" \
  --data-binary '{}'
```

Import CAO par cURL

Ce deuxième exemple prépare le fichier STEP livré avec le dépôt et crée un nouveau calcul volumique. Il réutilise les variables précédentes.

```
"$MAILLEUR_PY" - <<'PY'
import base64
import json
from pathlib import Path
source = Path("examples/assemblage_deux_volumes.step")
Path("output_api/client/upload.json").write_text(json.dumps({
    "filename": source.name,
    "content_base64": base64.b64encode(source.read_bytes()).decode("ascii")
}), encoding="utf-8")
PY
curl --fail-with-body -sS "$MAILLEUR_URL/api/uploads" \
  -H 'Content-Type: application/json' -H "X-Atelier-Token: $MAILLEUR_TOKEN" \
  --data-binary @output_api/client/upload.json -o output_api/client/uploaded.json

"$MAILLEUR_PY" - <<'PY'
import json
from pathlib import Path
upload = json.loads(Path("output_api/client/uploaded.json").read_text(encoding="utf-8"))
project = {
    "schema_version": 1, "name": "STEP par API", "mode": "volume",
```

```
    "geometry": {"kind": "import"},
    "mesh": {"size": 0.8, "min_size": 0.2, "threads": 1, "max_elements": 200000},
    "units": {"input": "mm", "internal": "m"}
}
Path("output_api/client/import_request.json").write_text(
    json.dumps({"project": project, "upload_id": upload["id"]}), encoding="utf-8")
PY
curl --fail-with-body -sS "$MAILLEUR_URL/api/jobs" \
  -H 'Content-Type: application/json' -H "X-Atelier-Token: $MAILLEUR_TOKEN" \
  --data-binary @output_api/client/import_request.json
```

Le dernier résultat contient un **nouvel identifiant de calcul** à suivre et utiliser pour les téléchargements.

5. Client Python complet, bibliothèque standard uniquement

Ce programme fonctionne depuis la racine du dépôt. Il nécessite le serveur démarré, mais aucune bibliothèque HTTP supplémentaire. Par défaut il maille `examples/atelier_plate.json`. Pour le STEP d'exemple, modifier la première ligne d'invocation en `.venv/bin/python - --source examples/assemblage_deux_volumes.step <<'PY'`. `Ctrl+C` demande l'annulation du calcul créé.

```
.venv/bin/python - <<'PY'
import argparse
import base64
import json
from pathlib import Path
import shutil
import time
from urllib.error import HTTPError
from urllib.request import Request, urlopen


class MailleurClient:
    def __init__(self, base="http://127.0.0.1:8765"):
        self.base = base.rstrip("/")
        self.token = self.request("/api/info")["token"]

    def request(self, route, body=None):
        headers = {}
        data = None
        if body is not None:
            data = json.dumps(body, allow_nan=False).encode("utf-8")
            headers = {"Content-Type": "application/json", "X-Atelier-Token": self.token}
        request = Request(self.base + route, data=data, headers=headers)
        try:
            with urlopen(request, timeout=30) as response:
                data = response.read()
                return json.loads(data) if response.headers.get_content_type() == "application/json" else data
        except HTTPError as error:
            message = error.read().decode("utf-8", errors="replace")
            raise RuntimeError(f"HTTP {error.code}: {message}") from error

    def upload(self, source):
        source = Path(source)
        size = source.stat().st_size
        if not 0 < size <= 50 * 1024 * 1024:
            raise ValueError("La source doit contenir de 1 octet à 50 Mio")
        return self.request("/api/uploads", {
            "filename": source.name,
            "content_base64": base64.b64encode(source.read_bytes()).decode("ascii")
        })["id"]

    def wait(self, identifier, timeout=330):
        deadline = time.monotonic() + timeout
        previous = None
        while time.monotonic() < deadline:
            job = self.request(f"/api/jobs/{identifier}")
            state = (job["status"], job.get("message"))
            if state != previous:
                print(state[0], state[1] or "", flush=True)
                previous = state
            if job["status"] == "completed":
                return job
            if job["status"] in ("failed", "cancelled"):
                raise RuntimeError(job.get("message", job["status"]))
            time.sleep(0.5)
```

```
        raise TimeoutError("Attente cliente expirée ; le calcul peut encore être actif")

    def cancel(self, identifier):
        return self.request(f"/api/jobs/{identifier}/cancel", {})

    def download(self, route, destination):
        destination = Path(destination)
        destination.parent.mkdir(parents=True, exist_ok=True)
        temporary = destination.with_name(destination.name + ".part")
        try:
            with urlopen(self.base + route, timeout=60) as response, temporary.open("wb") as output:
                shutil.copyfileobj(response, output, length=1024 * 1024)
            temporary.replace(destination)
        except Exception:
            temporary.unlink(missing_ok=True)
            raise
        return destination


parser = argparse.ArgumentParser()
parser.add_argument("--source", type=Path)
args = parser.parse_args()
client = MailleurClient()
project = json.loads(Path("examples/atelier_plate.json").read_text(encoding="utf-8"))
body = {"project": project, "use_advisor": False}
if args.source:
    body["upload_id"] = client.upload(args.source)
    project.update(name="CAO par API", mode="volume", geometry={"kind": "import"})

created = client.request("/api/jobs", body)
identifier = created["id"]
print("Identifiant :", identifier)
try:
    job = client.wait(identifier)
    destination = client.download(f"/api/jobs/{identifier}/bundle", "output_api/client_python/exports.zip")
    print("Rapport :", json.dumps(job["report"], ensure_ascii=False, indent=2))
    print("Exports :", destination.resolve())
except (KeyboardInterrupt, TimeoutError):
    print("Annulation :", client.cancel(identifier)["status"])
    raise
PY
```

Pour un serveur redémarré, créer une nouvelle instance du client afin d'obtenir le nouveau jeton. Le délai du client et celui du serveur sont indépendants. Une fermeture brutale du client ne stoppe pas automatiquement le calcul côté serveur.

6. Utiliser directement le pont Python

Le pont est pratique pour un script local ou une application Python. Il ne nécessite pas de serveur HTTP ni de jeton, mais les dépendances et le noyau compilé doivent être disponibles. Il normalise le projet, gère les dossiers, les processus, les unités et les conversions. Les calculs restent délégués à Rust et aux bibliothèques natives.

```
.venv/bin/python - <<'PY'
import json
from pathlib import Path
from atelier.native import NativeSession

project = json.loads(Path("examples/atelier_plate.json").read_text(encoding="utf-8"))
session = NativeSession(Path("output_api/direct"), timeout=300)
result = session.generate(project)
archive = session.export(
    Path(result["directory"]), Path("output_api/direct/exports.zip"), "bundle",
    units=project.get("units", {}).get("input", "mm"), planar=project.get("mode", "planar") == "planar"
)
print(json.dumps(result["report"], ensure_ascii=False, indent=2))
print(archive["file"])
PY
```

Pour une CAO, employer un projet `geometry.kind="import"` et `session.generate(project, source=Path("piece.step"))`. Le fichier source est copié dans le dossier de session. Il n'est pas inscrit comme chemin dans le projet portable.

Méthode de `NativeSession`	Usage
`inspect(project, source=None)`	Examiner la géométrie ; la tessellation d'affichage obtenue ne constitue pas un maillage de calcul exportable
`generate(project, source=None, use_advisor=False)`	Générer un maillage ; si `project.learning.enabled` vaut vrai, déléguer à `learn`
`learn(project, source=None, max_trials=12, epochs=100, seed=42, use_memory=True, max_seconds=240, memory_dir=None)`	Explorer des recettes, évaluer et mémoriser les expériences ; utilise les contrôles de qualité du projet
`memory_info(memory_dir=None)`	Lire les compteurs de la mémoire
`remesh(project, source=None, previous_directory=None)`	Reprendre un maillage existant ; géométrie, mode et unités doivent rester compatibles
`train(project=None, epochs=200, seed=42, cases=None)`	Entraîner le guide historique sur les cas fournis ou les démonstrations
`corpus_plan`, `corpus_status`, `corpus_preview`, `corpus_run`, `corpus_train`	Piloter le corpus par lots, sa reprise et son modèle à 128 caractéristiques
`export(directory, output, format_name, units="mm", planar=False)`	Exporter une session admissible ; `format_name="bundle"` produit tous les formats
`cancel()`	Demander l'arrêt du traitement de cette session ; appelable depuis un autre fil de l'application

Une instance n'accepte qu'une opération active à la fois. Utiliser des instances et des workspaces distincts pour des calculs indépendants. Les fichiers de modèle et la mémoire partagés restent des ressources à coordonner ; multiplier les workers ne garantit pas que plusieurs entraînements concurrents dans la même mémoire soient souhaitables.

La classe constitue une API Python de projet, sans engagement de stabilité entre versions. Pour une intégration durable, conserver la version du dépôt et un exemple de projet de référence avec le client.

7. Protocole du programme Rust

Le programme est un **exécutable lancé pour une requête**, pas un serveur persistant. Il accepte :

```
mailleur-core --version
mailleur-core --request /chemin/vers/requete.json
```

Le chemin de requête seul est également accepté. Une requête est un objet JSON contenant notamment `operation` et `output_dir`. Les calculs utilisant Gmsh nécessitent `gmsh_library`, chemin de sa bibliothèque partagée. Un maillage demande aussi `project` et, pour un import, `source`. `NativeSession` renseigne ces champs automatiquement.

Exemple exécutable de l'échange brut, depuis la racine :

```
.venv/bin/python - <<'PY'
import json
from pathlib import Path
import subprocess
from atelier.native import core_path, gmsh_library

output = Path("output_api/protocole_rust").resolve()
output.mkdir(parents=True, exist_ok=True)
request = {
    "operation": "mesh",
    "output_dir": str(output),
    "gmsh_library": str(gmsh_library()),
    "project": json.loads(Path("examples/atelier_plate.json").read_text(encoding="utf-8")),
    "response_mode": "summary"
}
request_file = output / "request.json"
```

```
request_file.write_text(json.dumps(request, allow_nan=False), encoding="utf-8")
subprocess.run([str(core_path()), "--request", str(request_file)], check=True)
print((output / "report.json").read_text(encoding="utf-8"))
PY
```

Utiliser des chemins absolus lors d'un appel brut : les chemins relatifs sont interprétés depuis le répertoire courant du processus. Le pont Python ajoute notamment les normalisations d'unités et de projet ; le contourner exige de respecter le contrat natif. Dans l'exemple, le projet fourni déclare explicitement ses unités.

Sur succès, la sortie standard contient `{"ok":true,"result":...}`. Avec `response_mode="summary"`, elle contient seulement `{"ok":true}` : les fichiers sont la source de résultat à lire. Le code de sortie vaut 0. Sur échec, un message JSON `{"ok":false,"error":"..."}` est écrit sur la sortie d'erreur, avec le code de sortie 2. Les diagnostics natifs éventuels peuvent précéder cette réponse ; ne pas considérer toutes les lignes du journal comme un unique document JSON.

Opérations natives	Rôle
`inspect`, `mesh`, `remesh`	Géométrie, maillage et reprise locale
`learn`, `memory_info`	Recherche et mémoire persistante
`train`	Guide historique
`corpus_plan`, `corpus_status`, `corpus_preview`, `corpus_run`, `corpus_train`	Planification, production et apprentissage du corpus
`metrics`	Mesures Rust sur un aperçu de maillage
`advisor_features`, `advisor_choose`, `advisor_default_cases`	Descripteurs, choix du guide historique et cas d'exemple
`rescale`	Conversion native des coordonnées d'un maillage
`export_unv`	Export UNV natif et contrôle par relecture

Les calculs géométriques produisent généralement `mesh.msh`, `model.brep`, `preview.json` et `report.json` selon l'opération. Le pont ajoute notamment `session.json`, la source copiée et les fichiers d'export. Certaines opérations utilitaires, telles que `rescale` et `export_unv`, renvoient leur résultat dans la réponse et ne suivent pas exactement ce jeu de fichiers. Consulter `rust_core/src/main.rs` et `atelier/native.py` avant d'ajouter une opération brute.

8. États, erreurs et diagnostic

Les états sont `queued`, `running`, `completed`, `failed`, `cancelled`. Une réponse **202 signifie accepté dans la file**, sans garantir que le calcul aboutira. Toujours attendre l'état final et lire son `message`. Un format particulier peut aussi échouer à la conversion : examiner `files` et les statuts du manifeste d'export, même si le calcul est `completed`.

Situation	Réponse ou comportement	Action
Jeton absent ou ancien, hôte/origine refusé	HTTP 403, JSON `error`	Relire `/api/info`, garder la même adresse et le même port
JSON invalide, clés répétées, NaN/Inf, mauvais réglage, source absente	HTTP 400, JSON `error`	Corriger la requête ; utiliser un objet JSON et les extensions admises
Identifiant ou route introuvable	HTTP 404, ou 400 si l'identifiant est malformé	Utiliser l'identifiant retourné, long de 32 caractères hexadécimaux
File de 16 calculs pleine ou entraînement déjà actif	HTTP 400	Attendre un créneau ou annuler un calcul
Export demandé avant la fin	HTTP 400	Attendre `completed`

Situation	Réponse ou comportement	Action
Gmsh ou noyau absent, géométrie refusée, qualité insuffisante	Calcul `failed`, avec `message`	Lancer `doctor`, consulter les journaux et revoir les paramètres
Temps global dépassé	Calcul `failed`, processus arrêtés	Réduire la densité ou augmenter `--timeout` selon le besoin
Arrêt précédent pendant un calcul	Au redémarrage, calcul reclassé `failed`	Relancer la requête ; la file HTTP ne reprend pas un maillage au milieu de son exécution

Les annulations arrêtent le processus de calcul et ses enfants. Elles ne suppriment pas l'historique ni les fichiers déjà produits. Les artefacts d'un essai rejeté peuvent servir au diagnostic ; ils ne doivent pas être utilisés pour contourner la décision de qualité. Les exports publics sont bloqués si le rapport refuse le maillage.

Pour diagnostiquer un calcul HTTP, examiner `jobs/{id}/calculation.log`, `job.json`, `request.json`, `error.json` s'il existe, puis les rapports et journaux dans `native/`. Certains chemins y sont absolus : les retirer ou les adapter avant de partager ces fichiers. Les données de démonstration et le code peuvent être publiés séparément des sources CAO privées et de la mémoire d'expériences.

9. Faire évoluer une intégration

Ajouter un type géométrique exige de coordonner validation Python, traitement Rust et exemples. Ajouter une opération exige aussi de définir le schéma de requête, les fichiers produits, les erreurs et les contrôles de qualité. La couche HTTP n'expose volontairement qu'une partie des opérations natives présentes ; documenter toute nouvelle route et conserver les routes existantes si des clients les utilisent.

Pour vérifier cette couche, les tests concernés sont `tests/test_server.py`, `tests/test_server_learning.py` et `tests/test_installation.py`. Les tests HTTP ouvrent uniquement une socket locale. Les essais de calcul réels nécessitent les dépendances et un noyau compilé pour la machine courante. Les tests de transport et d'installation emploient également des processus ou des substituts contrôlés pour vérifier les erreurs, l'annulation et la portabilité.

Annexe C — Tailles et capacités

Relevé du 4 novembre 2026, fondé sur les fichiers de l'archive **Atelier_Mailleur_Rust_Python_rc5.zip** récupérée depuis la bibliothèque du projet. Les chiffres de cette première section décrivent cette archive d'origine, avant les modifications locales de documentation et de code. Les autres sections indiquent les tailles des exemples réellement présents et les hypothèses d'extrapolation.

Le logiciel et ses modèles appris sont petits ; les historiques et les maillages conservés occupent l'essentiel de l'espace. Un million de configurations ne signifie pas un modèle d'intelligence artificielle contenant un million de pièces.

Unités et méthode

- 1 ko = 1 000 octets ; 1 Mo = 1 000 000 octets ; 1 Go = 1 000 000 000 octets.
- 1 Kio = 1 024 octets ; 1 Mio = 1 048 576 octets ; 1 Gio = 1 073 741 824 octets.
- Les tailles mesurées sont les longueurs des fichiers, hors blocs du système de fichiers. Elles ne sont ni la mémoire vive consommée ni les téléchargements nécessaires à l'installation.
- Le ZIP est compressé. Une fois extrait, ses fichiers occupent davantage d'octets. Les environnements Python, caches et fichiers de compilation créés ensuite constituent un poste supplémentaire.

Le script fourni permet de refaire le relevé après toute modification :

```
python3 scripts/inventory_sizes.py
python3 scripts/inventory_sizes.py --json
```

Il ne suit pas les liens symboliques et ne lit pas les contenus des corpus. Il sépare `.venv`, `.bootstrap`, `.runtime`, `.toolchain`, `target`, caches, sorties `output`/`outputs`, paquets et historique Git des fichiers du projet. Les données de validation de `verification/` restent comptées dans le projet et sont identifiées séparément. Un fichier JSON de relevé peut être conservé hors du dossier mesuré pour ne pas s'inclure lui-même dans le relevé suivant.

Taille de l'archive rc5 récupérée

Ensemble	Taille mesurée
Archive ZIP, 1 327 fichiers	20 319 571 octets = **20,32 Mo / 19,38 Mio**
Fichiers extraits, avant installation	74 315 966 octets = **74,32 Mo / 70,87 Mio**
Données, rapports et sources de référence `verification/`	70 298 468 octets = **70,30 Mo**
Ensemble hors `verification/`	4 017 498 octets = **4,02 Mo**

Environ **94,6 %** des octets décompressés proviennent de `verification/`. Ce dossier contient des preuves de validation, des essais, des maillages, des sources CAO, des copies de modèles et les sources rc4 servant aux comparaisons. Il n'est pas nécessaire de tout joindre à chaque envoi d'un modèle appris. Le conserver dans une livraison de référence facilite cependant la vérification des résultats annoncés.

Poste de l'archive d'origine	Octets	Rôle
`atelier/*.py` : 12 fichiers	221 139	Interface, transport vers Rust, orchestration et échanges de fichiers
`mailleur/*.py` : 7 fichiers	16 659	Commandes et compatibilité
`scripts/*.py` : 8 fichiers	75 614	Installation, lancement, validation, mesures et emballage

Poste de l'archive d'origine	Octets	Rôle
`rust_core/src/*.rs` : 13 fichiers	508 331	Moteur, qualité, corpus, entraînement et prédiction
`rust_core/Cargo.toml` et `Cargo.lock`	5 679	Configuration et versions de dépendances Rust
Binaire livré `rust_core/bin/linux-x86_64/mailleur-core`	2 471 456	Exécutable **Linux x86_64**, incompatible directement avec le Mac M3
Ressources d'interface de `atelier/`, hors Python	106 755	Feuille Qt, HTML, CSS et JavaScript
`tests/`	200 100	19 fichiers Python exécutables et références textuelles retirées
`examples/` : 22 fichiers	62 449	Projets et géométrie d'exemple, dont un STEP de 32 520 octets
`docs/`	85 374	Huit documents Markdown techniques
`README.md` et `NOUVEAUTES.md`	30 788	Présentation et historique
`MANIFEST.sha256`	229 986	Empreintes de la livraison d'origine

Ces lignes détaillent les postes principaux ; des petits lanceurs et configurations complètent le total. Les fichiers Markdown et scripts ajoutés par l'audit local augmentent légèrement ces chiffres. Le manifeste d'origine ne certifie pas les fichiers modifiés après extraction : il faut produire un nouveau manifeste lors d'une nouvelle livraison.

Ce que signifie « modèle »

Le projet manipule trois objets différents :

Objet	Exemple	À quoi il sert
Géométrie ou projet	`.step`, `.brep`, `.amp`, configuration JSON	Décrit la pièce et les réglages à mailler
Maillage calculé	`.msh`, `.unv`, `.inp`	Contient les nœuds, éléments et groupes pour le calcul
Modèle appris	`model.json`, `corpus.json`, `guide.json`	Contient les poids d'un petit réseau et ses métadonnées de validation

Le réseau conseille l'ordre ou le choix de recettes. **Gmsh/OpenCASCADE, piloté par Rust, construit la géométrie et le maillage.** Les poids seuls ne contiennent ni le moteur Gmsh, ni un maillage prêt à calculer, ni toutes les géométries d'entraînement.

Taille des réseaux et des fichiers appris

Les poids sont stockés dans du JSON avec leurs dimensions, leur schéma de caractéristiques, leurs métriques et leur provenance. Ils ne sont pas livrés en PyTorch ou safetensors dans cette rc5.

Réseau présent dans les sources rc5	Architecture	Paramètres	Poids seuls en f64, calcul théorique	Fichier réel de démonstration
Guide historique de choix d'algorithme	8 entrées → 12 neurones → 3 sorties	147	1 176 octets	`guide.json` : **13 869 octets**
Guide de mémoire interactive, schéma v2	128 → 12 → 1	1 561	12 488 octets	`experience/model.json` : **49 964 octets**

Réseau présent dans les sources rc5	Architecture	Paramètres	Poids seuls en f64, calcul théorique	Fichier réel de démonstration
Guide entraîné sur le corpus	128 → 32 → 1 par défaut	4 161	33 288 octets	`models/current.json` ou copie `models/corpus.json` : **125 562 octets**
Point de reprise du corpus	État permettant de reprendre l'entraînement	—	—	`training.checkpoint.json` : **371 034 octets**

Les tailles réelles ci-dessus sont mesurées dans les preuves récentes livrées, pas estimées à partir d'un million de cas. Le fichier du modèle de corpus existe à plusieurs emplacements sous forme de copies de même taille. Le point de reprise contient plus d'état qu'un fichier de prédiction et n'est pas nécessaire pour utiliser simplement les poids.

Pour un réseau 128 → H → 1, le nombre de paramètres vaut `128×H + H + H + 1 = 130×H + 1`. Le volume de poids dépend donc surtout de la largeur du réseau. Il ne croît pas proportionnellement au nombre d'exemples. Les métadonnées de provenance peuvent croître, et les versions successives conservées occupent un espace cumulatif.

Les neuf fichiers `experience/models/model-000001.json` à `model-000009.json` totalisent **444 623 octets**. Ce sont neuf sauvegardes successives du même type de guide, pas neuf spécialistes de géométries indépendants. Les anciens guides interactifs v1 de 16 entrées et 217 paramètres apparaissent encore dans les preuves rc3 : ils ne doivent pas être confondus avec le schéma v2 de production.

L'état final des démonstrations récentes garde le classement neuronal **désactivé** (`deploy_enabled=false`) : aucun gain de validation suffisant n'est démontré. Cela n'empêche pas la génération classique ni la recherche de recettes et leur réemploi revalidé. Envoyer ces fichiers à une autre personne ne transforme pas ces essais en modèle industriel déjà validé.

Corpus : taille réelle et extrapolation au million

Le plan cible du rapport livré est de **250 000 cas structurés + 750 000 aléatoires**. Il n'a encore exécuté aucun de ces cas. Ses trois fichiers de métadonnées représentent seulement **5 058 octets**, d'après `verification/corpus/summary.json`. Définir un plan d'un million ne remplit pas le disque avec un million de pièces.

Le pilote réellement exécuté contient **48 configurations**, réparties sur 24 familles, et **96 résultats de stratégies**. Il a produit un maillage dans 46 cas ; 45 cas disposent d'au moins une stratégie admissible. Ses segments JSONL récents représentent **666 890 octets**, soit **13 893,54 octets par configuration en moyenne**, avec deux stratégies par configuration et sans conservation de tous les maillages.

Hypothèse strictement proportionnelle à ce pilote	JSONL estimé seul
1 000 configurations	13,89 Mo
10 000 configurations	138,94 Mo
100 000 configurations	1,389 Go
250 000 configurations	3,473 Go
750 000 configurations	10,420 Go
1 000 000 configurations	**13,894 Go / 12,94 Gio**

Cette extrapolation est un **ordre de grandeur**, pas une limite garantie : le pilote est petit, ses pièces sont simples, ses cas invalides ont des lignes plus courtes et il ne teste que deux stratégies par cas. Le plan habituel prévoit quatre stratégies ; les lignes seront alors plus longues, sans qu'on puisse multiplier exactement par deux puisque les données communes à la configuration ne sont pas dupliquées de la même façon. Pour dimensionner une vraie campagne, mesurer un lot représentatif de plusieurs centaines ou milliers de cas avec les réglages définitifs.

Le dossier complet du pilote occupe **1 298 409 octets**, dont les segments 666 890 et les trois fichiers de modèle/reprise 622 158. Ajouter les copies de modèles installées dans les espaces de travail, les journaux, rapports et fichiers temporaires au budget d'une campagne. L'entraînement lit les segments par mini-lots ; cela évite de charger tout le corpus en mémoire, mais pas son stockage ni ses relectures.

Si tous les maillages sont conservés

L'option `retain_meshes` change fortement le budget. Le rapport de mesure rc5 `verification/optimization/bulk.json` utilise un cas de 45 183 tétraèdres pour lequel les artefacts lourds représentent environ **10,40 Mo par évaluation**. À titre purement arithmétique, un million de cas × quatre stratégies × 10,40 Mo représente **41,6 To**, hors inspections de référence, corpus, copies et journaux. Ce scénario n'est pas une prévision de la taille des futures pièces.

La configuration normale d'une campagne ne conserve pas tous ces maillages. Elle garde les paramètres, caractéristiques, qualités et échecs nécessaires à l'apprentissage. Une stratégie de conservation sélective des cas difficiles et des résultats de référence est plus adaptée à une campagne longue.

Taille de la mémoire interactive

Le dossier de démonstration récent `verification/continuous_learning/continuous_runs/20261104T091837738488/experience/` occupe **1 428 396 octets** pour six géométries, neuf contextes et 29 observations/journalisations :

Contenu	Octets
`events.jsonl` : historique détaillé des essais	593 472
`memory.json` : index des recettes et contextes	307 817
`model.json` : guide courant	49 964
`models/` : neuf versions	444 623
`sources/` : source STEP conservée	32 520
Total	**1 428 396**

La taille d'une source CAO importée peut dépasser largement cet exemple. Répéter une pièce ne supprime pas la nécessité de conserver le journal des essais. Les limites documentées de l'index interactif sont 1 000 contextes, 32 000 observations et 128 Mio ; celle du journal est 512 Mio. Elles ne plafonnent pas les octets de tous les maillages, sources ou modèles. Une limite atteinte déclenche une erreur explicite, sans purge automatique. Le corpus segmenté constitue un mécanisme distinct, prévu pour de plus grands volumes.

Ce qu'il faut envoyer à une autre personne

Besoin du destinataire	Contenu nécessaire
Utiliser le logiciel	Sources et ressources du projet, installation des dépendances et binaire compatible avec son système, ou compilation locale
Utiliser un guide de corpus entraîné	JSON complet du guide compatible, avec son schéma et ses métriques ; dans son espace de travail : `models/corpus.json`
Utiliser le guide historique	JSON compatible dans `models/guide.json`, selon les limites de domaine du guide
Reprendre l'expérience sur les mêmes pièces	Dossier `experience/` cohérent : index, journal, sources et guide, en respectant les noms et empreintes
Reprendre un entraînement de corpus	Segments, manifeste, progression et modèle/point de reprise compatibles ; conserver les jeux de validation et test
Échanger une pièce et ses réglages	Projet portable `.amp` et fichiers nécessaires selon son mode de sauvegarde
Faire tourner un calcul mécanique	Maillage exporté, groupes et unités contrôlés, plus le modèle mécanique propre au solveur

Arrêter les calculs qui écrivent avant de copier un dossier d'expérience ou un corpus. Copier un ensemble cohérent dans un nouvel espace de travail évite de remplacer un historique existant par accident. Un guide courant pèse ici environ **50 ko** ou **126 ko** ; transmettre tout le corpus pour une simple prédiction n'est pas nécessaire. Pour un usage reproductible, joindre la version du logiciel, le schéma de caractéristiques, les réglages, la provenance des données et les métriques. Conserver `deploy_enabled` et ses conditions d'activation : ne pas forcer ce champ pour contourner la validation.

Les données de mémoire et de corpus peuvent contenir les géométries, noms de pièces et réglages de leurs auteurs. Vérifier le contenu choisi avant une diffusion publique. Un dépôt GitHub peut contenir le code et la documentation ; les gros corpus et campagnes se distribuent séparément. Cette rc5 doit aussi être accompagnée d'une licence explicite et du bilan de ses dépendances avant d'annoncer les droits de redistribution ; voir THIRD_PARTY.md.

Produire un petit paquet de poids vérifiable

`scripts/share_model.py` crée un ZIP contenant le JSON du modèle, un guide d'installation et un manifeste d'empreintes SHA-256. Il fonctionne avec la bibliothèque standard Python, sans Gmsh ni interface. Les trois types sont `historique`, `experience` et `corpus` :

```
python3 scripts/share_model.py --kind corpus \
  --model MON_CORPUS/models/current.json \
  --output output/mon_guide_corpus.zip

python3 scripts/share_model.py --kind experience \
  --model MON_ESPACE/experience/model.json \
  --output output/mon_guide_experience.zip

python3 scripts/share_model.py --kind historique \
  --model MON_ESPACE/models/guide.json \
  --output output/mon_guide_historique.zip
```

Le script refuse un fichier incomplet, un point de reprise présenté comme modèle, un ancien schéma incompatible, des poids non finis ou hors plage, des dimensions erronées et une activation contredisant les preuves déclarées. Un modèle valide mais désactivé peut être partagé : il reste désactivé. Ces contrôles de format et de cohérence ne sont pas une certification de performance ; le noyau Rust reste responsable de ses propres contrôles au chargement et de la validation de chaque maillage.

Le script ne joint aucun corpus, dossier mémoire ou fichier CAO séparé. Pour le guide historique, il retire également `training_configs`, champ qui peut embarquer les géométries d'entraînement et qui n'est pas requis par son chargeur

de prédiction. Les poids, les preuves de validation et la décision d'activation sont conservés. Le manifeste signale ce retrait et garde l'empreinte du fichier source. L'option facultative `--report CHEMIN/rapport.json` joint un rapport choisi explicitement ; vérifier son contenu, car il peut contenir des noms et chemins locaux. Un fichier ZIP existant n'est jamais écrasé.

Les paquets de démonstration produits pendant l'audit sont disponibles sous `output/partage/`. Ils transportent les petits guides des essais livrés, tous désactivés dans leur état final. Pour le guide interactif, les poids seuls ne suffisent pas à reprendre l'historique cumulatif : les groupes de la mémoire d'origine doivent encore être présents dans les observations rejouées. Le paquet ne constitue donc pas un import de mémoire complète.

Espace d'installation et mémoire de l'ordinateur

Les **74,32 Mo** décompressés ne comprennent pas l'environnement Python complet ni les caches Cargo. Qt/PySide6, Gmsh/OpenCASCADE, NumPy et les autres dépendances représentent un poste nettement plus gros que les scripts. Leur taille exacte dépend des versions, du système et des roues téléchargées ; la mesurer après installation avec `scripts/inventory_sizes.py` dans la catégorie des environnements. La compilation Rust crée `rust_core/target/`, distinct du binaire final. Les caches externes au dossier du projet ne sont pas mesurés par ce script.

La mémoire vive de **16 Go** du Mac M3 est largement supérieure aux poids des petits réseaux ci-dessus. La géométrie, les tableaux de nœuds/éléments, les contrôles de qualité et l'affichage sont les postes à mesurer sur de grosses pièces. La taille JSONL totale du corpus n'a pas à tenir dans la RAM grâce à la lecture progressive. Cela ne garantit pas qu'une pièce particulière de très grande taille tiendra en mémoire : mesurer le pic de mémoire et la durée sur des cas représentatifs avant une campagne longue.

Références internes : rapport du corpus, apprentissage continu, optimisations rc5, guide du corpus et des langages, limites.